Multi-Agent Architecture

https://github.com/aiyshwary/multi-agent-revenue-churn

Python LLM Multi-Agent Pandas Docker GitHub Actions

OVERVIEW

A production-grade agentic pipeline that combines LLM-driven planning with deterministic Python analytics to perform revenue analysis and churn prediction, featuring an Orchestrator-Executor-Reflection loop with memory management and Docker deployment.

IMPACT

Designed and implemented a multi-agent revenue and churn analysis system where an LLM Planner coordinates an Executor, Validator, and Reflection agent, keeping LLMs focused on reasoning while Python handles all numeric computation.

TECHNICAL WALKTHROUGH

It's easiest to think of it as a deterministic data-processing pipeline wrapped in a flexible multi-agent orchestration layer.
LLMs are used for planning, validation and reflection, but they never touch the raw numbers – Python/pandas do all of the work.

Overall run-loop

Every execution is a closed-loop coordinated by the Orchestrator

i) Planner – produces an ordered list (or graph) of actions.

ii) Executor – carries out each action, writing results to short-term memory.

iii) Validator – runs deterministic checks (and optionally an LLM review).

iv) Reflection – inspects outputs and suggests next steps (retry, re-plan, investigate).

v) Orchestrator – enforces retries, idempotency, circuit-breaking, and

vi) logs everything.

"LLMs propose & critique; Python is the source of truth."

Core components & their roles

Component Responsible for
PlannerAgent / LLMPlanner Translate objectives into steps.

Can be deterministic or
LLM-assisted.

ExecutorAgent Implements the actual data

i) transformations (load,

ii) assign_quarter, aggregate, churn,

validate, reflect).
ValidatorAgent / LLMValidator Checks totals, churn sanity, etc.

Optionally asks an LLM to review
outputs.

ReflectionAgent Scores results and returns
suggestions (“investigate lost
clients” , “retry aggregation” , etc.).

MemoryManager Tracks short-term step outputs,

i) compresses summaries to

ii) long-term memory, maintains

iii) idempotency and a tiny semantic

index.

CircuitBreaker Stops repeating failing steps after
a threshold.

MetricsCollector Simple in-process counters &
timers written to metrics.json.

GraphRunner Executes a dependency graph,
honouring depends_on and allowing
parallelism.

SemanticMemory Deterministic vector store used by
agents/LLMs for retrieval.

Tools package

Deterministic helpers called by executors

i) DataLoader – handles CSV/Excel loading, normalises wide-format

ii) sheets. assign_quarter, Aggregator, ChurnCalculator – compute quarterly and client-level aggregates.

iii) Validator – sanity-checks and totals match.

iv) SemanticMemory – local, reproducible embedding store.

v) MemoryManager – higher-level wrapper combining short/long/

vi) semantic memory.

vii) Execution workflow in code

viii) Initialization

ix) Orchestrator.__init__ wires up agents, optional per-role LLM clients, circuit breaker, memory, metrics, and idempotency state.

x) Planning

xi) planner.plan() returns a list of dicts; LLMPlanner can override with an LLM result.

Example node

i) "id": "agg_q1",

ii) "action": "aggregate_quarter_region",

iii) "depends_on": ["load"],

iv) "validation_checkpoint": "validate_quarterly"

v) Graph execution

vi) GraphRunner.run() respects explicit depends_on and sequence order.

vii) Ready nodes may run in parallel (bounded by max_workers).

viii) Each node dispatches to Orchestrator._run_single_step().

ix) Single-step handling

x) Idempotency key computed from step name + hashed context.

xi) Circuit breaker checked.

xii) Retries with exponential backoff

xiii) (configurable max_retries, backoff_base).

xiv) Success logs, metrics, and memory updates are recorded.

xv) Validation steps trigger ValidatorAgent; failures can invoke the LLM validator.

xvi) Post-run reflection & output

xvii) Reflection suggestions gathered.

xviii) Metrics and logs are dumped to outputs.

xix) Visualizations generated (churn trends, quarterly totals, lost-client plots).

xx) Memory model

xxi) Short-term – per-step summaries kept in memory during a run.

xxii) Long-term – compressed summaries persisted to memory.json; idempotency flags, failure counts.

xxiii) Semantic – optional vector store (semantic_memory.json); deterministic embeddings support small retrievals for prompt context.

xxiv) Memory managers are called throughout

xxv) (store_step_summary, mark_executed, etc.); tests cover both plain and semantic variants.

xxvi) Reliability & observability features

xxvii) Retries & backoff – every step wrapped with a retry loop.

xxviii) Circuit breaker – after a configurable number of failures, a step is

xxix) “open” and future attempts abort.

xxx) Idempotency – prevents repeated side-effects when a step is re-executed with the same input.

xxxi) Metrics – counters and timings recorded; persisted to JSON.

xxxii) Logs – per-step status, errors, attempts stored in self.log.

xxxiii) Tests – numerous unit tests simulate failures, chunking, parallel runs, planner budgets, LLM clients, etc.

xxxiv) Multi-agent collaboration

The “multi-agent” aspect is largely architectural

i) Separation of concerns – planning, executing, validating, reflecting, memory, LLM interaction are separate agents/objects.

ii) Pluggability – LLMs can be swapped in per role (planner, validator, reflection).

iii) Deterministic fallbacks – every LLM-based agent has a pure-Python alternative.

iv) GraphRunner lets agents' actions be composed into a directed acyclic graph with dependencies.

v) Semantic memory is shared between agents for context.

vi) Agents communicate via the context dict and the MemoryManager; the Orchestrator orchestrates, but the implementation of each step lives in its own class.

vii) Sample sequence

viii) PlannerAgent.plan() [{"step":"load_data"},

ix) {"step":"assign_quarter"}, …]

Orchestrator runs load_data

i) reads CSV/Excel,

ii) stores df in context,

iii) logs row count & revenue sum. assign_quarter adds a "Quarter" column. aggregate_quarter_region groups and writes a CSV. client_quarterly pivots per-client revenue. compute_churn compares successive quarters and dumps JSON. validate checks totals match. reflect inspects churn results and notes "significant_loss" .

iv) Orchestrator persists metrics & visualizations, returns final result.

v) With LLMs enabled, the planner/validator/reflection may interject suggestions or even replan entire flows if the LLM advises.

vi) Workflow highlights from tests

vii) Chunking (with/without parallel workers) produces identical churn reports.

viii) GraphRunner correctly respects dependencies and parallelism.

ix) Circuit breaker trips after repeated failures.

x) LLMPlanner respects token budgets and falls back when necessary.

xi) Semantic memory returns stable results.

xii) Orchestrator accepts per-role LLM clients and still runs normally.

xiii) Visual/Output artifacts

The run writes to the outputs directory

i) quarterly_by_region.csv, client_quarterly_revenue.csv

ii) churn_report.json (plus timestamped variants)

iii) metrics.json, memory.json, semantic_memory.json

iv) Visualizations (.png, .html) for churn and revenue trends.

v) Final note

vi) This architecture is intentionally deterministic, testable, and transparent.

vii) The multi-agent terminology describes a modular design where separate classes (“agents”) own discrete responsibilities and communicate via shared state under orchestration.

viii) The workflow is a classic Plan → Execute → Validate → Reflect → Act loop, with control flow managed by the Orchestrator and optional LLMs providing higher-level reasoning within a tightly-scoped numerical system.

KEY DETAILS

1. Planner emits an executable plan (list or graph of nodes) from a high-level task description.

2. Orchestrator runs each step via the Executor, applies deterministic validators, and uses Reflection to decide retry or replan actions.

3. Memory Manager persists short summaries and semantic snippets for context retrieval across long pipelines.

4. Outputs churn_report.json, per-client quarterly revenue CSVs, and HTML/PNG visualizations per run.

5. Fully containerized with Docker Compose; CI/CD via GitHub Actions with pytest unit test suite.

6. GraphRunner resolves step dependencies as a DAG and executes ready nodes in parallel via ThreadPoolExecutor, with per-step CircuitBreaker and SHA-256 idempotency keys to prevent duplicate execution.

7. SemanticMemory uses a dependency-free in-memory cosine-similarity index with deterministic MD5/SHA-256 token hashing for fully reproducible tests; long-term memory uses zlib compression and persists to JSON.

8. LLMClient supports both a mock provider (deterministic stubs for offline testing) and OpenAI provider (gpt-3.5-turbo); TokenEstimator enforces budgets before every call. Visualizations generated with Matplotlib (PNG) and Plotly (interactive HTML).